Desarrollo de Aplicación Híbridas

UT2: Fundamentos de Ionic y Angular

UT2: Fundamentos de Ionic y Angular

¡Bienvenido/a a la Unidad 2! Ahora que ya tienes tu entorno de desarrollo preparado, es el momento de sumergirnos en el corazón de nuestras aplicaciones. En esta unidad, aprenderás los fundamentos de Angular, el framework que da vida a tus proyectos en Ionic.

A lo largo de esta unidad, usaremos como ejemplo la creación de una sencilla **aplicación de lista de tareas** para aplicar todos los conceptos. Empezaremos a crear componentes interactivos, mostrar datos dinámicos y a dar forma a tu proyecto personal.

Objetivos de Aprendizaje

Al finalizar esta unidad, serás capaz de:

- Entender la estructura de carpetas de un proyecto Ionic.
- Comprender los conceptos básicos de **TypeScript**.
- Entender la **arquitectura de componentes** de Angular y cómo generar los tuyos.
- Conectar la lógica de tu aplicación con la vista usando **Data Binding**.
- Manipular la estructura de tus páginas con **Directivas Estructurales**.
- Crear tu primer componente personalizado y funcional para tu proyecto personal.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.1. La Estructura de un Proyecto Ionic

2.1. La Estructura de un Proyecto Ionic

Cuando creas un proyecto con `ionic start`, se genera una estructura de carpetas y archivos que puede parecer abrumadora al principio. No te preocupes, no necesitas conocer cada archivo. Vamos a centrarnos en los directorios y ficheros con los que trabajarás el 99% del tiempo.

El Corazón de la App: La Carpeta `/src`

Casi todo tu trabajo como desarrollador se realizará dentro de la carpeta `/src` (source/código fuente).

- **`/src/app`**: Es el directorio más importante. Contiene toda la lógica, las páginas y los componentes de tu aplicación.
  - `/app-routing.module.ts`: El "mapa" de tu aplicación. Define las rutas (URLs) y qué página se debe cargar para cada una.
  - `/app.component.ts`: El componente raíz de toda la aplicación. Es como el marco principal que contiene todo lo demás.
  - `/app.module.ts`: El registro central de tu app. Aquí se declaran todas las páginas y componentes para que Angular sepa que existen.
- **`/src/assets`**: Aquí guardarás todos los recursos estáticos como imágenes, iconos, fuentes o archivos JSON.
- **`/src/theme`**: Contiene el archivo `variables.scss`, donde puedes definir y modificar la paleta de colores global de tu aplicación para que todos los componentes de Ionic (botones, cabeceras, etc.) sigan tu guía de estilo.
- **`/src/index.html`**: Es la única página HTML "real" de toda la aplicación. Es un cascarón vacío donde Ionic/Angular inyectará dinámicamente todos tus componentes y páginas. Normalmente no necesitarás tocarla.

Otras Carpetas Relevantes

- **`/node_modules`**: Este directorio contiene todas las librerías y dependencias externas que tu proyecto necesita para funcionar (Ionic, Angular, etc.). ¡Nunca modifiques nada aquí

directamente! Se gestiona automáticamente con NPM.
- **`/www`**: Cuando compilas tu aplicación para producción, Ionic genera en esta carpeta la versión final optimizada de tu código (HTML, CSS y JavaScript) que se subirá a un servidor o se empaquetará en una app nativa. No debes editar esta carpeta manualmente.
- **`ionic.config.json`**: Un archivo de configuración para Ionic, donde se define el nombre de tu proyecto y el tipo de integración (Angular).

El Cerebro de las Dependencias: `package.json`

Junto a las carpetas, en la raíz de tu proyecto, encontrarás el archivo **`package.json`**. Este fichero es como el DNI de tu proyecto. Contiene metadatos importantes y, lo más crucial, una lista de todas las librerías externas de las que depende tu aplicación.

- **`"dependencies"`**: Es la lista de paquetes necesarios para que tu aplicación funcione en producción (ej: Angular, Ionic).
- **`"devDependencies"`**: Es la lista de paquetes que solo se necesitan para desarrollar, pero no en la app final (ej: herramientas de testing o de análisis de código).

El Rol de `npm install`

Cuando te descargas un proyecto de un repositorio (como GitHub), la carpeta `node_modules` nunca se incluye porque pesa muchísimo. En su lugar, solo viene el archivo `package.json`.

El comando `npm install` lo que hace es leer la lista de "dependencies" y "devDependencies" del `package.json` y descargar todas esas librerías desde internet, creando la carpeta `node_modules` en tu ordenador. Es el paso que "da vida" a un proyecto descargado.

¿Cuándo lo modificarás?

Rara vez editarás este archivo a mano. Se actualiza automáticamente cuando añades una nueva librería. Por ejemplo, si en el futuro necesitas una librería para manejar fechas, ejecutarías en la terminal:

```
npm install date-fns
```

Este comando hace dos cosas: 1. Descarga el paquete `date-fns` y lo guarda en `node_modules`. 2. Añade automáticamente `date-fns` a la lista de "dependencies" en tu archivo `package.json`.

Otros Archivos de Configuración Clave

Además de los ya mencionados, existen otros archivos en la raíz del proyecto que actúan como centros de control. No los modificarás a menudo, pero es fundamental saber para qué sirven.

`angular.json` - El Centro de Control

Este es el fichero de configuración principal del CLI de Angular. Define cómo se construye, se prueba y se sirve tu aplicación. Si necesitaras añadir librerías globales de CSS o scripts a todo tu proyecto, este sería el lugar para registrarlas.

`capacitor.config.ts` - El DNI de tu App Nativa

Este archivo es el corazón de Capacitor. Aquí se definen datos cruciales para la app que se instalará en el móvil, como el identificador único (`appId`), el nombre de la app (`appName`) y la carpeta donde se encuentra el código web (`webDir`).

`tsconfig.json` - Las Reglas del Juego

Este es el archivo de configuración para el compilador de TypeScript. Le da las instrucciones sobre cómo "traducir" tu código TypeScript (.ts) a JavaScript (.js), la versión de JavaScript a la que debe apuntar y qué nivel de rigurosidad debe aplicar al revisar tu código.

`.gitignore` - La Lista de Exclusión

Un fichero de texto que le dice a Git (el sistema de control de versiones) qué archivos y carpetas debe ignorar y no subir nunca al repositorio. Su función más importante es evitar que se suba la gigantesca carpeta `node_modules`.

Conocer esta estructura te ayudará a navegar por tu proyecto con confianza y a saber dónde crear o buscar cada tipo de archivo.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.2. TypeScript, el Superpoder de JavaScript

2.2. TypeScript, el Superpoder de JavaScript

Aunque Ionic usa tecnologías web estándar, la lógica de nuestros componentes no se escribe en JavaScript normal, sino en **TypeScript**. Piensa en TypeScript como una versión de JavaScript "con superpoderes".

Su principal superpoder es el **Tipado Estático**.

¿Qué es el Tipado Estático?

En JavaScript normal, una variable puede contener cualquier tipo de dato, y puede cambiar en cualquier momento:

```
let miVariable = "Hola";      // Es un texto (string)
miVariable = 100;             // Ahora es un número (number)
miVariable = true;            // Y ahora un booleano (boolean). ¡No hay problema
```

TypeScript nos obliga a ser más estrictos y a definir qué tipo de dato va a contener una variable, lo que nos ayuda a evitar muchísimos errores. Si intentas asignar un tipo de dato incorrecto, el editor de código (VS Code) te avisará inmediatamente.

```
let nombre: string = "Ana";
let edad: number = 28;
let esEstudiante: boolean = true;
let notas: number[] = [8, 9, 7.5];
let aficiones: string[] = ["Leer", "Programar"];
```

Una Aclaración Clave: ¿Cuándo se usa `let`?

La palabra clave `let` se utiliza para declarar variables que van a existir dentro de un bloque de código concreto, como el interior de una función o un método. Sin embargo, cuando declaramos datos que pertenecen a todo un componente y que queremos que estén disponibles para su plantilla HTML, los declaramos como **propiedades de la clase**, y en ese caso **no se usa `let`**.

- **Usa `let`:** Para variables temporales dentro de un método.
- **No uses `let`:** Para las propiedades principales de un componente (las que están directamente dentro de la clase).

Lo verás más claro en los siguientes ejemplos.

Interfaces: Creando Nuestros Propios Tipos

A veces, nuestros datos son más complejos que un simple número o texto. Para nuestra lista de tareas, cada tarea tendrá un `id`, un `titulo` y un estado `completada`. TypeScript nos permite crear un "molde" o un "contrato" para estos objetos usando una **interfaz**.

Paso 1: Generar el archivo de la interfaz

Para mantener nuestro proyecto organizado, crearemos las interfaces en una carpeta específica. Usamos la CLI de Ionic para que nos genere el archivo automáticamente:

```
ionic generate interface interfaces/tarea
```

Paso 2: Definir la estructura de la interfaz

Ahora, abrimos el nuevo archivo `src/app/interfaces/tarea.interface.ts` y definimos la estructura que debe tener cualquier objeto que quiera ser del tipo `Tarea`.

```
// Dentro de src/app/interfaces/tarea.interface.ts
export interface Tarea {
  id: number;
  titulo: string;
  completada: boolean;
}
```

Paso 3: Usar nuestra nueva interfaz

Una vez definida la interfaz, podemos usarla en nuestros componentes para definir los datos que manejará la vista. Las variables que contienen los datos de un componente se declaran como **propiedades de su clase** (y por tanto, sin `let`).

Por ejemplo, si quisiéramos tener una única tarea en nuestra página de inicio, editaríamos el archivo `home.page.ts` de la siguiente manera:

```
// Dentro de src/app/home/home.page.ts
import { Component } from '@angular/core';
import { Tarea } from '../interfaces/tarea'; // Importamos el molde

@Component({
  selector: 'app-home',
  // ...
})
export class HomePage {

  // Declaramos 'miPrimeraTarea' como una propiedad de la clase HomePage (sin
  miPrimeraTarea: Tarea = {
    id: 1,
    titulo: "Aprender TypeScript",
    completada: false
  };

  constructor() {}

}
```

Sin embargo, en nuestra aplicación no tendremos una sola tarea, sino una **lista de tareas**. Por lo tanto, lo que realmente haremos será declarar una propiedad que sea un **array de Tareas**. Esta es la estructura que usaremos en las siguientes secciones:

```
// Dentro de src/app/home/home.page.ts
import { Component } from '@angular/core';
import { Tarea } from '../interfaces/tarea';

@Component({
```

```
  selector: 'app-home',
  // ...
})
export class HomePage {

  // Esta es la propiedad que realmente usaremos (sin let)
  listaDeTareas: Tarea[] = [
    { id: 1, titulo: "Comprar el pan", completada: true },
    { id: 2, titulo: "Estudiar componentes de Angular", completada: false },
    { id: 3, titulo: "Hacer el reto de la UT2", completada: false }
  ];

  constructor() {}

}
```

Al declarar `listaDeTareas` como una propiedad de la clase `HomePage`, estará disponible para ser usada en su plantilla HTML (`home.page.html`), como veremos en las siguientes secciones.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.3. Componentes: Los Ladrillos de tu App

2.3. Componentes: Los Ladrillos de tu App

La idea principal de Angular, y de todos los frameworks modernos, es la **arquitectura de componentes**. En lugar de pensar en una gran página HTML, dividimos la interfaz en pequeñas piezas independientes y reutilizables, como si fueran ladrillos de LEGO.

¿Qué es un Componente?

Un componente es la combinación de tres cosas:

- **Una plantilla (HTML):** La estructura visual de esa pieza.
- **Una lógica (TypeScript):** El comportamiento y los datos de esa pieza.
- **Unos estilos (CSS/SCSS):** La apariencia de esa pieza.

Paso 1: Generar el componente

Primero, usamos la CLI de Ionic para crear la estructura de nuestro componente. En la terminal, ejecutamos:

```
ionic generate component components/task-item
```

Esto creará una carpeta `components/task-item` con los archivos necesarios.

Standalone vs. NgModules: El Gran Cambio de Angular

Antes de configurar nuestro componente, es crucial entender un cambio fundamental en las versiones modernas de Angular. Existen dos formas de gestionar los componentes:

- **El Enfoque Clásico (NgModules):** Antes, cada componente debía ser "declarado" dentro de un archivo especial llamado "módulo" (un fichero `.module.ts`). Este módulo actuaba como una caja que agrupaba varios componentes y herramientas.
- **El Enfoque Moderno (Standalone):** Ahora, un componente puede ser **independiente**. No necesita pertenecer a ninguna "caja". En su lugar, el propio componente declara explícitamente qué herramientas (otros componentes, directivas, etc.) necesita para funcionar.

En este curso, usaremos siempre el enfoque moderno `standalone` porque es más sencillo, más eficiente y es el estándar actual en los nuevos proyectos de Ionic.

Paso 2: Configurar nuestro Componente como Standalone

Cuando generamos un componente, la CLI no añade `standalone: true` por defecto. Debemos hacerlo nosotros manualmente. Este es el código inicial que define la lógica de nuestro `task-item.component.ts`.

```
// Contenido de: src/app/components/task-item/task-item.component.ts

import { Component, Input } from '@angular/core';
import { Tarea } from '../../interfaces/tarea';

@Component({
  selector: 'app-task-item',
  templateUrl: './task-item.component.html',
  styleUrls: ['./task-item.component.scss'],
  standalone: true, // ¡La línea clave! Le decimos a Angular que este componer
  imports: [] // Dejamos el array de imports vacío por ahora. Lo llenaremos er
})
export class TaskItemComponent {

  @Input() tarea!: Tarea;

  constructor() { }

  mostrarDetalles() {
    console.log('Datos de la tarea:', this.tarea);
  }

}
```

Desglosando la Lógica del Componente

Analicemos las partes más importantes del archivo `.ts`:

- **`@Input() tarea!: Tarea;`**: El decorador `@Input()` convierte la propiedad `tarea` en una "puerta de entrada". Permite que un componente padre (como nuestra página `home`) le pase datos a este componente hijo. El signo de exclamación `!` es una promesa que le hacemos a TypeScript, asegurándole que esta propiedad siempre recibirá un valor desde fuera.
- **`constructor() { }`**: Es el primer método que se ejecuta al crear una instancia del componente. Su uso principal, que veremos más adelante, es la "inyección de dependencias", es decir, pedir los servicios que el componente necesita para funcionar.
- **`mostrarDetalles() { ... }`**: Este es un método personalizado que hemos creado nosotros. A diferencia de los anteriores, no se ejecuta automáticamente. Podemos llamarlo desde la plantilla HTML, por ejemplo, al hacer clic en un botón, para que realice una acción específica.

Entendiendo el Decorador `@Component` Standalone

- **`selector: 'app-task-item'`**: Esto no cambia. Sigue siendo el nombre de nuestra nueva etiqueta HTML.
- **`standalone: true`**: Esta es la propiedad que convierte nuestro componente en una pieza independiente. Es fundamental añadirla.
- **`imports: []`**: Este array se convierte en el más importante. Aquí debemos listar **todas y cada una de las "herramientas"** que nuestra plantilla HTML (`task-item.component.html`) va a utilizar. De momento lo dejamos vacío, pero en la siguiente sección, cuando construyamos la vista, tendremos que llenarlo con los componentes de Ionic que necesitemos.

En la siguiente sección sobre "Data Binding" veremos el contenido del archivo `task-item.component.html` y cómo se conecta con la lógica que hemos definido aquí.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.4. Data Binding: Conectando Lógica y Vista

2.4. Data Binding: Conectando Lógica y Vista

El "Data Binding" o "Enlace de Datos" es el pegamento mágico que conecta la lógica de tu componente (el archivo `.ts`) con su vista (el archivo `.html`). Es lo que permite que la interfaz de usuario reaccione automáticamente a los cambios en los datos, y viceversa.

En esta sección, construiremos la plantilla HTML de nuestro `TaskItemComponent` y veremos cómo usar el Data Binding para mostrar la información que recibe desde su propiedad `@Input()`.

> ### Importante: Componentes Standalone y sus Dependencias
>
> Antes de escribir el HTML, debemos recordar que nuestro `TaskItemComponent` es **standalone** (independiente). Esto significa que no conoce ninguna etiqueta de Ionic (como `<ion-card>`, `<ion-item>`, etc.) a menos que se las importemos explícitamente en su archivo `.ts`.
>
> Para que la plantilla que vamos a crear funcione, debemos asegurarnos de que el archivo `task-item.component.ts` importa todos los módulos y componentes que necesita:

```
// Contenido de: src/app/components/task-item/task-item.component.ts

import { Component, Input } from '@angular/core';
import { Tarea } from '../../interfaces/tarea';
// 1. Importamos los componentes de Ionic que usaremos en el HTML
import { IonCard, IonItem, IonCheckbox, IonLabel, IonButton } from '@ioni
// 2. Importamos CommonModule para poder usar directivas como [class.tach
import { CommonModule } from '@angular/common';

@Component({
```

```
    selector: 'app-task-item',
    templateUrl: './task-item.component.html',
    styleUrls: ['./task-item.component.scss'],
    standalone: true,
    // 3. Registramos los módulos y componentes en el array 'imports'
    imports: [CommonModule, IonCard, IonItem, IonCheckbox, IonLabel, IonBut
  })
  export class TaskItemComponent {

    @Input() tarea!: Tarea;

    constructor() { }

    mostrarDetalles() {
      console.log('Datos de la tarea:', this.tarea);
    }
  }
```

Hacer esto a mano puede ser un poco tedioso. No te preocupes, en la **siguiente sección (2.5)** te presentaremos una herramienta que automatiza este proceso por ti.

La Vista: `task-item.component.html`

Ahora sí, este es el código que añadiremos a nuestro archivo de la plantilla para mostrar una tarea individual.

```
<!-- Contenido de: src/app/components/task-item/task-item.component.html -->

<ion-card>
  <ion-item>
    <ion-checkbox
      slot="start"
      [checked]="tarea.completada">
    </ion-checkbox>

    <ion-label [class.tachado]="tarea.completada">
      {{ tarea.titulo }}
    </ion-label>
```

```
      <ion-button (click)="mostrarDetalles()" slot="end">
        Ver
      </ion-button>
    </ion-item>
  </ion-card>
```

Desglosando los Componentes de Ionic

En la plantilla de nuestro componente hemos usado varias etiquetas de Ionic para construir la interfaz. Es importante que entiendas qué hace cada una:

- **`<ion-card>`**: Es un contenedor con un estilo predefinido (sombra, bordes redondeados) ideal para agrupar información relacionada, como los datos de nuestra tarea.
- **`<ion-item>`**: Es cada una de las filas o elementos que van dentro de un contenedor más grande (como una tarjeta o una lista). Organiza el contenido en una línea y es muy versátil.
- **`<ion-checkbox>`**: Es el componente de casilla de verificación. Lo usamos para representar el estado `completada` de la tarea. La propiedad `slot="start"` le dice que se coloque al principio del `ion-item`.
- **`<ion-label>`**: Es la etiqueta de texto optimizada de Ionic. Se usa dentro de un `ion-item` para mostrar el texto principal, como el título de nuestra tarea.
- **`<ion-button>`**: Es el componente de botón. Ionic le aplica estilos para que se vea y se comporte de forma nativa en iOS y Android. La propiedad `slot="end"` le dice que se coloque al final del `ion-item`.

Desglosando el Data Binding

Angular nos ofrece cuatro formas de realizar este enlace:

1. Interpolación `{{ }}`

Es la forma más sencilla. Se usa para mostrar el valor de una propiedad como texto en el HTML. En nuestro ejemplo, la usamos para mostrar el título de la tarea:

```
<ion-label>{{ tarea.titulo }}</ion-label>
```

2. Property Binding `[ ]`

Se usa para asignar un valor a una propiedad de un elemento HTML. El nombre de la propiedad se encierra entre corchetes. Lo usamos para dos cosas:

- Para cambiar el color de la tarjeta: `[color]="tarea.completada ? 'light' : 'primary'"`. Si la tarea está completada, el color será 'light'; si no, será 'primary'.
- Para marcar o desmarcar el checkbox: `[checked]="tarea.completada"`.

Angular también nos da una sintaxis especial para enlazar clases CSS:

```
<ion-label [class.tachado]="tarea.completada">
```

Esta línea significa: "Aplica la clase CSS `.tachado` a este `ion-label` solo si la propiedad `tarea.completada` es `true`".

3. Event Binding `( )`

Se usa para ejecutar un método del componente cuando ocurre un evento en el HTML (como un clic). El nombre del evento se encierra entre paréntesis.

```
<ion-button (click)="mostrarDetalles()">Ver</ion-button>
```

Esta línea significa: "Cuando el usuario haga clic en este botón, ejecuta el método `mostrarDetalles()` que está en el archivo `.ts`".

4. Two-Way Binding `[( )]`

Es una combinación de los dos anteriores que se usa principalmente en formularios para mantener sincronizado un input con una variable. Lo veremos en profundidad más adelante.

> #### ¿Dónde Veo el Resultado del `console.log`?
>
> Al hacer clic en el botón "Ver", se ejecutará el método `mostrarDetalles()`, que contiene un `console.log()`. Para ver su salida, necesitas abrir las **Herramientas de Desarrollador** de tu navegador (normalmente con la tecla F12 o haciendo clic derecho -> "Inspeccionar") y seleccionar la pestaña **"Consola"**.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.5. Herramienta Esencial: Angular Language Service

2.5. Herramienta Esencial: Angular Language Service

Al trabajar con las versiones modernas de Ionic y Angular, pronto te darás cuenta de un detalle importante: los componentes se crean como **"standalone"** (independientes). Esto significa que cada componente es como una isla: no conoce ninguna etiqueta de Ionic (como `<ion-button>` o `<ion-card>`) a menos que se la importemos explícitamente.

Hacer esto a mano puede ser tedioso y es fácil olvidar alguna importación, lo que provoca errores como `'ion-button' is not a known element`.

Afortunadamente, existe una herramienta que nos soluciona este problema de forma casi mágica: la extensión **Angular Language Service** para Visual Studio Code.

> ### ¿Qué hace esta extensión?
>
> Esta extensión oficial de Angular le da "superpoderes" a tu editor. Entiende perfectamente cómo se conectan tus archivos `.ts` y `.html`, y te ofrece ayudas cruciales:
>
> - **Detecta errores en tiempo real:** Si usas un componente en tu HTML que no has importado en el `.ts`, te lo marcará con un subrayado rojo.
> - **Ofrece soluciones automáticas (Quick Fix):** ¡Esta es la magia! Te permite solucionar el error con un solo clic, añadiendo automáticamente la línea de `import` que falta.

Instalación (Obligatoria)

Para seguir el curso de forma fluida, es imprescindible que instales esta extensión antes de continuar con la creación de componentes.

1. Abre Visual Studio Code.
2. Ve a la pestaña de **Extensiones** en la barra lateral (el icono de los cuatro cuadrados).
3. En el buscador, escribe `Angular Language Service`.
4. El primer resultado, creado por "Angular", es el que necesitas. Haz clic en **Instalar**.

Una vez instalada, es posible que necesites reiniciar VS Code para que se active por completo.

¿Cómo funciona en la práctica?

Imagina que en un archivo `.html` escribes la etiqueta `<ion-button>`. Como aún no la has importado en el archivo `.ts` correspondiente, la extensión te la subrayará como un error.

Ahora, simplemente tienes que:

1. Poner el cursor del ratón sobre la etiqueta con el error.
2. Hacer clic en el enlace **"Quick Fix..."** (o pulsar `Ctrl + .` o `Cmd + .`).
3. Seleccionar la primera opción, que será algo como **"Import IonButton from @ionic/angular/standalone"**.

¡Listo! La extensión añadirá automáticamente la línea `import { IonButton } from '@ionic/angular/standalone';` al principio de tu archivo `.ts` y también la registrará en el array `imports` del `@Component`. El error desaparecerá.

A partir de ahora, cada vez que uses un nuevo componente de Ionic en tu HTML, solo tienes que seguir este sencillo paso para que tu código funcione correctamente.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.5. Directivas: Dando Vida a tus Vistas

2.5. Directivas: Dando Vida a tus Vistas

Las directivas son instrucciones especiales que podemos añadir a nuestro HTML para que Angular manipule el DOM (la estructura de la página). Las dos directivas estructurales más importantes y que usarás constantemente son `*ngIf` y `*ngFor`.

`*ngIf`: Muestra u oculta elementos

La directiva `*ngIf` es como una sentencia `if` de programación, pero en tu plantilla HTML. Añade o elimina un elemento del DOM basándose en si una condición es verdadera (`true`) o falsa (`false`).

En nuestra app de lista de tareas, podemos usarla para mostrar un mensaje amigable cuando la lista de tareas esté vacía.

`*ngFor`: Repite elementos para cada ítem de una lista

La directiva `*ngFor` es el equivalente a un bucle `for`. Itera sobre un array y repite un bloque de HTML por cada elemento de ese array.

Esta es la directiva que usaremos para renderizar nuestra lista de tareas. Por cada objeto `Tarea` en nuestro array `listaDeTareas`, crearemos una instancia de nuestro componente `<app-task-item>`.

Aplicando las Directivas en la Página Principal

Para que todo esto funcione, necesitamos actualizar dos archivos en nuestra página `home`.

1. La Lógica en `home.page.ts`

Nos aseguramos de que el componente `HomePage` importe todo lo que su plantilla va a necesitar: los componentes de Ionic, nuestro `TaskItemComponent` y el `CommonModule` (que contiene las directivas `*ngIf` y `*ngFor`).

```
// Contenido de: src/app/home/home.page.ts

// Imports básicos de Angular y Ionic
import { Component } from '@angular/core';
import { IonHeader, IonToolbar, IonTitle, IonContent } from '@ionic/angular/st

// Imports de nuestra aplicación
import { Tarea } from '../interfaces/tarea'; // Nuestra interfaz de datos
import { TaskItemComponent } from '../components/task-item/task-item.component
import { CommonModule } from '@angular/common'; // Módulo necesario para usar

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  // Al ser un componente standalone, debemos importar aquí todo lo que usemos
  imports: [
    IonHeader, IonToolbar, IonTitle, IonContent, // Componentes de Ionic
    TaskItemComponent, // Nuestro componente hijo
    CommonModule // El módulo para las directivas
  ],
})
export class HomePage {

  // Esta es la lista de datos que mostraremos en la vista.
  // Es un array de objetos que deben cumplir la estructura de la interfaz 'Ta
  listaDeTareas: Tarea[] = [
    { id: 1, titulo: "Comprar el pan", completada: true },
    { id: 2, titulo: "Estudiar componentes de Angular", completada: false },
    { id: 3, titulo: "Hacer el reto de la UT2", completada: false }
  ];

  constructor() {}
}
```

2. La Vista en `home.page.html`

Ahora, en la plantilla, usamos la estructura de cabecera con efecto de colapso, `*ngIf` para la condición de lista vacía y `*ngFor` para repetir nuestro componente.

```
<!-- Contenido de: src/app/home/home.page.html -->

<!-- La cabecera principal, visible en la parte superior de la pantalla -->
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      Lista de Tareas
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
  <!-- Esta segunda cabecera crea el efecto de título grande que se encoge al
  <ion-header collapse="condense">
    <ion-toolbar>
      <ion-title size="large">Lista de Tareas</ion-title>
    </ion-toolbar>
  </ion-header>

  <!-- Directiva *ngIf: este div solo existirá en el DOM si la condición es ve
  <div *ngIf="listaDeTareas.length === 0" class="mensaje-vacio">
    <p>¡Enhorabuena!</p>
    <p>No tienes tareas pendientes.</p>
  </div>

  <!--
    Directiva *ngFor: este es el bucle.
    - Por cada objeto 'tarea' que haya en el array 'listaDeTareas'...
    - ...Angular creará una instancia de nuestro componente <app-task-item>.
  -->
  <app-task-item
    *ngFor="let tarea of listaDeTareas"
    [tarea]="tarea"> <!-- Property Binding: le pasamos el objeto 'tarea' actua
  </app-task-item>
</ion-content>
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.6. Reto: Aplica los Fundamentos en tu Proyecto

2.6. Reto: Aplica los Fundamentos en tu Proyecto

¡Es el momento de dar el primer gran paso en tu proyecto personal! En esta tarea, aplicarás los conceptos fundamentales de Angular que hemos visto en esta unidad para empezar a construir la estructura de tu aplicación.

Esta tarea se divide en dos niveles: un reto básico obligatorio para aprobar, y varios retos de mejora que serán necesarios para poder optar a las calificaciones más altas.

> ### ✅ Reto Básico (Requisitos Mínimos para Aprobar)
>
> El objetivo es construir una estructura de componentes funcional que muestre una lista de datos estáticos relacionados con la temática de tu proyecto. Para ello, debes:
>
> 1. **Crear tu Interfaz de Datos:** Usando el comando `ionic generate interface`, crea una interfaz que defina la estructura del elemento principal de tu app (ej: `Producto`, `Noticia`, `Receta`, etc.).
> 2. **Crear un Componente Reutilizable:** Usando `ionic generate component`, crea un componente "hijo" que sirva para mostrar un único elemento de tu lista (ej: `producto-item`, `noticia-card`).
> 3. **Configurar el `@Input()`:** En el archivo `.ts` de tu nuevo componente, define una propiedad con el decorador `@Input()` para que pueda recibir los datos del elemento que debe mostrar.
> 4. **Maquetar el Componente Hijo:** En el archivo `.html` de tu componente, usa los componentes de Ionic que necesites (`ion-card`, `ion-item`, `ion-label`...) y el Data Binding (`{{ }}`, `[ ]`) para mostrar la información del objeto recibido por el `@Input`.
> 5. **Crear una Lista de Datos:** En el archivo `home.page.ts` (el "padre"), crea una propiedad que sea un array con varios objetos de ejemplo que sigan la estructura de tu interfaz.
> 6. **Mostrar la Lista:** En el archivo `home.page.html`, usa la directiva `*ngFor` para recorrer tu array de datos y renderizar una instancia de tu componente hijo por cada

elemento de la lista, pasándole los datos correspondientes a través del `@Input`.

Al completar este reto, tendrás la base de tu aplicación funcionando, mostrando una lista de datos de tu propia temática.

🚀 Retos para la Máxima Calificación (Bien / Excelente)

Completar el reto básico es suficiente para aprobar, pero para obtener una calificación de **Bien** o **Excelente**, se espera que demuestres iniciativa investigando e implementando una o varias de las siguientes mejoras en tu proyecto.

- **Mejora 1: Estilos Dinámicos.** Añade alguna propiedad extra a tu interfaz (ej: un booleano `enOferta` para un producto, o `esUrgente` para una noticia) y utiliza el *property binding* de clases `[class.mi-clase]="condicion"` o de colores `[color]="condicion ? 'success' : 'primary'"` para que la apariencia de tus elementos cambie dinámicamente según sus datos.
- **Mejora 2: Componente de Cabecera.** Crea un segundo componente más sencillo, por ejemplo, `app-header`. Este componente podría mostrar el título de tu aplicación y quizás un subtítulo o un logo. Luego, úsalo en tu `home.page.html`. Esto demostrará que has entendido perfectamente el concepto de componentización.
- **Mejora 3: Formateo con Pipes.** Investiga qué son los "Pipes" de Angular. Son una forma muy sencilla de transformar datos directamente en el HTML. Podrías investigar cómo usar el `DatePipe` para formatear una fecha, o el `UpperCasePipe` para mostrar un título en mayúsculas. ¡La documentación oficial de Angular es tu mejor amiga aquí!
- **Mejora 4: Manejo de Datos Opcionales.** Adapta tu componente hijo para que pueda manejar el caso en que no reciba datos. Para ello, deberás cambiar tu `@Input` para que sea de tipo `MiInterfaz | undefined` y usar la directiva `*ngIf` en la plantilla HTML para mostrar el contenido solo si los datos existen. Esto demuestra una programación más robusta.

Nota Importante: El objetivo de esta fase es mostrar datos de forma dinámica. La funcionalidad para añadir, editar o eliminar elementos la implementaremos en las próximas unidades.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

2.7. Recursos Adicionales

2.7. Recursos Adicionales

Los conceptos que hemos visto en esta unidad son la base de todo el desarrollo con Ionic y Angular. Si quieres profundizar o consultar la documentación oficial, aquí tienes algunos enlaces que te serán de gran utilidad:

- **Documentación oficial de TypeScript:**
  - Manual de TypeScript - Tipos del día a día (en inglés) <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html> : Ideal para entender los tipos básicos y las interfaces.
- **Guía oficial de Angular sobre Componentes:**
  - Introducción a los Componentes (en inglés) <https://angular.io/guide/component-overview> : La fuente de información principal sobre la arquitectura de componentes.
- **Guía oficial de Angular sobre Data Binding:**
  - Sintaxis de Enlace de Datos (en inglés) <https://angular.io/guide/binding-syntax> : Una explicación detallada de todos los tipos de binding.
- **Guía oficial de Angular sobre Directivas:**
  - Directivas Estructurales (en inglés) <https://angular.io/guide/built-in-directives> : Documentación oficial sobre *ngIf, *ngFor y *ngSwitch.
- **Componentes de la Interfaz de Ionic:**
  - Catálogo de Componentes de Ionic <https://ionicframework.com/docs/components> : Aquí puedes ver todos los componentes visuales que Ionic te ofrece (botones, tarjetas, listas, etc.) con ejemplos de código.

Consultar la documentación oficial es una habilidad clave en el mundo del desarrollo. ¡No dudes en explorarla!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**